

Utilizando *uma tupla*

A **tupla** é a irmã imutável da lista: guarda valores em ordem, mas não deixa alterar depois de criada. Nesta aula: como criar (e a pegadinha da tupla de um item só), o erro ao tentar alterar, desempacotamento, troca de valores, funções que devolvem vários valores, tuplas como chave de dicionário e quando escolher tupla ou lista.

()

A vírgula cria a tupla

Imutável

Desempacotar

a, b = b, a

.count() · .index()

Tupla × lista

Formação Claude Code 2026 · Baseado em docs.python.org (Tutorial: Tuples and Sequences; Built-in Types: Sequence Types), consultados em outubro de 2026 · Exemplos testados com Python 3.14 em `exemplos/` · Saídas reais

A vírgula *é que cria a tupla*

criar.py

```
ponto = (3, 4)
cores = ("vermelho", "verde", "azul")
sem_parenteses = 10, 20      # a vírgula é que cria a tupla
um_item = ("só",)           # um item: precisa da vírgula
nao_e_tupla = ("só")         # sem vírgula: é só uma str
print(type(sem_parenteses), type(um_item), type(nao_e_tupla))
print(cores[0], cores[-1], cores[1:])
print(len(cores), "verde" in cores)
```

SAÍDA

```
<class 'tuple'> <class 'tuple'> <class 'str'>
vermelho azul ('verde', 'azul')
3 True
```

- ✓ Parênteses, itens separados por vírgula
- ✓ Na verdade, quem cria a tupla é a **vírgula**: `10, 20` já é tupla
- ✓ Tupla de um item: `("só",)`, com a vírgula no fim
- ✓ Índices, fatias, `len()` e `in` funcionam como em listas e strings

`("só")` sem vírgula é só uma string entre parênteses: a saída mostra `<class 'str'>`.

Depois de criada, *não muda*

imutavel.py

```
cores = ("vermelho", "verde", "azul")
cores[0] = "amarelo"
```

```
Traceback (most recent call last):
  File "imutavel.py", line 2, in <module>
    cores[0] = "amarelo"
    ~~~~~^^
```

```
TypeError: 'tuple' object does not support item assignment
```

- ✓ Mesmo erro das strings (aula 07): **TypeError**, não aceita atribuição
- ✓ Tupla não tem `append`, `remove` nem `sort`
- ✓ Para "mudar", crie outra: `cores = ("amarelo",) + cores[1:]`

A imutabilidade é uma **garantia**: quem recebe uma tupla sabe que ninguém vai alterá-la no caminho. Coordenadas, datas, configurações fixas.

Um valor *para cada variável*

desempacotar.py

```
ponto = (3, 4)
x, y = ponto           # desempacotamento
print(x, y)
nome, idade, cidade = ("Ana", 29, "Recife")
print(f"{nome} ({idade}) mora em {cidade}")
primeiro, *resto = (1, 2, 3, 4)
print(primeiro, resto)  # resto vira lista
a, b = 1, 2
a, b = b, a             # troca usando tupla
print(a, b)
```

SAÍDA

```
3 4
Ana (29) mora em Recife
1 [2, 3, 4]
2 1
```

- ✓ `x, y = ponto` : cada variável recebe um item, na ordem
- ✓ A quantidade precisa bater; senão, **ValueError**
- ✓ `*resto` recolhe "o que sobrou" numa **lista**
- ✓ `a, b = b, a` : o lado direito vira uma tupla e é desempacotado (a troca da aula 06)

Devolvendo *vários valores*

retorno.py

```
def min_max(numeros):  
    return min(numeros), max(numeros)  # devolve uma tupla  
  
menor, maior = min_max([7, 2, 9, 4])  
print(menor, maior)  
resultado = min_max([7, 2, 9, 4])  
print(resultado, type(resultado))
```

SAÍDA

```
2 9  
(2, 9) <class 'tuple'>
```

- ✓ `return min(...), max(...)` devolve uma tupla
- ✓ Quem chama pode desempacotar direto: `menor, maior = ...`
- ✓ Funções (`def`) são o assunto da **aula 17**; aqui é uma prévia

É o mesmo padrão do `divmod()` da aula 06 e do `dividir()` do capítulo 7 do guia.

Poucos métodos, *muitas funções*

metodos.py

```
notas = (7, 9, 7, 10, 7)
print(notas.count(7))      # 3
print(notas.index(10))     # 3
print(sorted(notas))       # sorted devolve LISTA
print(tuple(sorted(notas)))
print(list(notas))         # vira lista para alterar
```

SAÍDA

```
3
3
[7, 7, 7, 9, 10]
(7, 7, 7, 9, 10)
[7, 9, 7, 10, 7]
```

- ✓ Só dois métodos: `.count()` e `.index()`
- ✓ `sorted()`, `max()`, `min()`, `sum()`, `len()` funcionam
- ✓ `sorted()` devolve uma **lista**; `tuple()` converte de volta
- ✓ `list(tupla)` e `tuple(lista)` convertem entre as duas

Chave de dicionário *e memória*

chave_dict.py

```
# Tuplas podem ser chave de dicionário; listas não
capitais = {(-8.05, -34.88): "Recife",
            (-23.55, -46.63): "São Paulo"}
print(capitais[(-8.05, -34.88)])
```

Recife

Chaves de dicionário (aula 12) e itens de conjunto (aula 11) precisam ser imutáveis. Tupla serve; lista dá **TypeError**.

tupla_vs_lista.py

```
import sys
lista = [1, 2, 3, 4, 5]
tupla = (1, 2, 3, 4, 5)
print(sys.getsizeof(lista), "bytes (lista)")
print(sys.getsizeof(tupla), "bytes (tupla)")
```

104 bytes (lista)
88 bytes (tupla)

Com os mesmos 5 itens, a tupla ocupou menos memória neste Python 3.14. Os números exatos variam entre versões e sistemas; a diferença é pequena, e a escolha deve vir do **significado**, não da economia.

Tupla *ou lista?*

	LISTA	TUPLA
Sintaxe	[1, 2, 3]	(1, 2, 3)
Alterar depois	Sim	Não
Métodos	Muitos: append, sort, pop...	Só count e index
Chave de dicionário	Não	Sim
Uso típico	Coleção que cresce: itens do carrinho, notas, tarefas	Registro fixo: ponto (x, y), data (dia, mês, ano), retorno de função

Regra de bolso do guia (cap. 8): use tuplas para dados que **não devem mudar** (coordenadas, configurações) e listas para coleções que mudam.

Distância *entre dois pontos*

Enunciado: com os pontos `a = (1, 2)` e `b = (4, 6)`, desempacote as coordenadas e calcule a distância: raiz de $(x_2 - x_1)^2 + (y_2 - y_1)^2$.

Uma resposta possível

```
# Exercício: distância entre dois pontos (tuplas)
a = (1, 2)
b = (4, 6)
x1, y1 = a
x2, y2 = b
distancia = ((x2 - x1) ** 2 + (y2 - y1) ** 2) ** 0.5
print(f"Distância entre {a} e {b}: {distancia}")
```

SAÍDA

Distância entre (1, 2) e (4, 6): 5.0

- ✓ `** 0.5` é a raiz quadrada (aula 06)
- ✓ 3, 4 e 5: o triângulo retângulo clássico
- ✓ Desafio: peça as coordenadas com `input()` e monte as tuplas

Da primeira linha às coleções

AULA	ASSUNTO	IDEIA CENTRAL
02	Primeiro programa	print(), rodar no terminal, ler erros pela última linha
03	Tipos de dados	Todo valor tem um tipo; conversões e ValueError
04	input()	Sempre devolve texto: converta
05	Concatenação	f-strings e formatos como :.2f
06	Operadores	// e %, precedência, and/or/not
07 e 08	Strings	Índices, fatias, imutáveis; métodos devolvem texto novo
09	Listas	Mutáveis; = não copia
10	Tuplas	Imutáveis; desempacotamento

Próximos passos do módulo: **set** e **dicionário** (aulas 11 e 12), depois **if** , **for** , **while** e funções.

Teste o que *ficou*

1. Qual destes é uma tupla de um item?

A) (5)

B) (5,)

C) [5]

D) {5}

B. A vírgula cria a tupla.

2. `t = (1, 2); t[0] = 9` resulta em:

A) (9, 2)

B) TypeError

C) IndexError

D) (1, 2, 9)

B. Tuplas são imutáveis.

3. Depois de `a, b = 1, 2` e `a, b = b, a`:

A) `a=1, b=2`

B) `a=2, b=1`

C) Erro

D) `a=2, b=2`

B. Troca por desempacotamento.

4. Quando preferir tupla a lista?

A) Lista que cresce

B) Registro fixo, como coordenadas

C) Para ordenar

D) Nunca

B. E para chaves de dicionário.

Valores juntos *e fixos*

A tupla é uma sequência **imutável**: criada pela vírgula, sem `append` nem `sort`, com só `.count()` e `.index()`. Brilha no **desempacotamento**, na troca `a, b = b, a`, no retorno de vários valores e como chave de dicionário.

("só",)

Imutável

x, y = ponto

*resto

Chave de dict

Próxima aula → 11 · Utilizando um Set

Fontes: docs.python.org/3/tutorial/datastructures.html#tuples-and-sequences; docs.python.org/3/library/stdtypes.html#tuples, consultados em outubro de 2026.
Exemplos em `exemplos/`, executados com Python 3.14.4.